Field Hours

CSCE 411 Design and Analysis of Algorithms

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

Asymptotics

Big-O says an upper bound. Theta says a tight bound. A loop from 1 to n that does constant work is Theta of n. A nested pair of such loops is Theta of n squared. Drop constants and lower terms when you write the bound, and do not drop them when you are choosing between two algorithms on a small n.

Name the model. These bounds count steps, not cache misses. CSCE 629 will be stricter about that sentence. You need the shapes first.

Three shapes.

n: one pass

n log n: a balanced divide

n squared: a loop inside a loop

A double loop

1. For i from 1 to n, for j from 1 to i, one step.

2. The inner loop runs 1 + 2 + ... + n times, which is n(n + 1) / 2.

3. That is Theta of n squared. The triangle is still quadratic.

Problem 1. Is 3n + 20 equal to O(n)? Is it Theta(n)?

Answer. Both. It is bounded above by a constant times n, and also below by a constant times n, for large n. O alone would also allow a looser claim such as O(n squared), which is true and useless.

Problem 2. Your Theta bound says n log n and the program is slow on n = 10. Is the bound wrong?

Answer. Not necessarily. The bound describes large n. A huge constant, or a setup cost, can dominate at 10. Measure the small case. Do not 'fix' a correct asymptotic claim by deleting it.

Lecture 2 · 50 minutes

Sorting

Insertion sort walks the array and inserts each new item into the sorted prefix. It is Theta of n squared in the worst case, and it is fast on a nearly sorted array because each insert walks only a short distance.

Merge sort splits in half, sorts, and merges. The merge of n items is n steps, and there are log n levels, so the bound is Theta of n log n. It needs extra space for the merge. Comparison sorts cannot beat n log n in the worst case, in the comparison model. That sentence has a model. Do not apply it to counting sort.

Merge sort on eight items.

One merge of 8

Two merges of 4

Four merges of 2

Each level costs 8

Eight items, three levels

1. Level cost is 8. Levels are log2(8) = 3.

2. Total merge steps are at most 24.

3. Insertion sort on a reversed array of 8 does about 8 times 7 / 2 comparisons, also around 28, and the gap grows after that. At n = 8 the constants still matter.

Problem 1. When would you choose insertion sort over merge sort?

Answer. On a tiny array, or a nearly sorted one, where the quadratic algorithm does little work and the merge's extra space and recursion do not pay. On a large reversed array, merge.

Problem 2. Counting sort of integers in a small range is linear in n plus the range. Why does that not break the n log n lower bound?

Answer. The lower bound is for sorting by comparisons. Counting sort uses the values as array indexes. Different model, different bound.

Lecture 3 · 50 minutes

Greedy

A greedy algorithm takes the local best step and does not reconsider. It is correct only with an argument, usually an exchange: any solution that differs from the greedy choice can be edited toward it without getting worse.

Interval scheduling, earliest finish first, is the example to memorize. Sort by end time. Take the first interval that does not overlap the one you took. A later start that finishes earlier leaves more room. Taking the shortest interval instead is a different rule and it fails on a simple picture.

Earliest finish.

1. Sort by end time

2. Take the first

3. Skip anything that overlaps it

4. Repeat

Three intervals

1. A runs from 0 to 10. B and C run from 1 to 3 and from 4 to 6.

2. Earliest finish takes B, then C. Two intervals.

3. Shortest-first might take A if you only look at a different set. On this set, shortest-first also takes B and C. Change A to be the unique short interval that blocks two longer ones and shortest-first fails. Earliest finish still works. The proof is the exchange, not one picture.

Problem 1. You sorted intervals by start time and took the first that fits. Give a counterexample shape.

Answer. One long interval that starts first, and two short ones that start later and do not overlap each other. Start-time greedy takes the long one and blocks both short ones. Earliest finish takes the two short ones.

Problem 2. Is 'always take the locally best' a proof?

Answer. No. It is a description of the algorithm. The proof shows that some optimal solution agrees with the choice, usually by exchanging. Without that, you have a heuristic.

Lecture 4 · 50 minutes

Dynamic programming

Dynamic programming applies when a problem splits into smaller problems that repeat. Compute the small ones once and store them. A recursive solution that recomputes the same call is the algorithm you are replacing.

The edit distance, or the longest common subsequence, fills a table. The cell for two prefixes depends on three neighbors. The table is the order that makes each neighbor ready. The graduate course will ask for the invariant in words. Write it now: the cell is the answer for those prefixes, not a guess.

A distance table.

Rows are one string

Columns are the other

Each cell is the prefixes above and to the left

Kitten and sitting, one cell

1. You do not need the whole famous example. Two letters, A and B, against A.

2. Cell (1, 1): the letters match, so the cost is the diagonal, 0.

3. Cell (1, 2): B against the end of A does not match. The cost is one plus the best of insert, delete, or substitute. The table, not the recursion tree, stores that.

Problem 1. A recursive fibonacci recomputes the same n many times. What do you store?

Answer. The answer for each k from 0 to n, in order. Each new value is the sum of the two previous stored values. The exponential tree becomes a linear loop.

Problem 2. What must be true of a cell before you use it in a later cell?

Answer. It is the finished answer for its subproblem, not a partial value. Fill in an order that respects the dependencies, usually smaller prefixes first.

Lecture 5 · 50 minutes

Shortest paths and spanning trees

Dijkstra's algorithm grows a set of vertices whose distances are final. It needs nonnegative weights. With a heap it runs in about (vertices plus edges) times log vertices. A negative edge can make a 'final' distance wrong, because a later path could be cheaper. Bellman-Ford handles negative edges, slowly, and detects a negative cycle by a leftover update.

A minimum spanning tree connects every vertex at least total weight. Kruskal adds the next lightest edge that does not form a cycle. The cut property is the proof: the lightest edge across any cut is safe.

Dijkstra, one step.

1. Take the unsettled vertex with the smallest distance

2. Relax its edges

3. That vertex is done, if weights are nonnegative

Three vertices

1. Edges A-B weight 1, A-C weight 4, B-C weight 1. Start at A.

2. Settle A at 0. Relax: B is 1, C is 4.

3. Settle B. Relax B-C: C becomes 2. The path A-B-C beats the direct edge. Settle C at 2.

Problem 1. Change B-C to weight -10. Why does the same algorithm fail if it already settled C at 4 before looking?

Answer. It may have marked C final at 4 via the direct edge, then the negative edge from B would want to improve it. Nonnegative weights are what make the first settlement final. Use Bellman-Ford, and check for a negative cycle if you can loop.

Problem 2. Kruskal sorts edges. What does the disjoint-set structure do?

Answer. It answers whether the two ends are already connected, in nearly constant time. If they are, the edge would cycle and you skip it. The sort is still the expensive part.

Lecture 6 · 50 minutes

A first look at NP

A decision problem is in P when some algorithm answers it in polynomial time. It is in NP when a proposed answer can be checked in polynomial time, even if finding it seems hard. A problem is NP-complete when it is in NP and every problem in NP reduces to it. The reduction is the proof technique. You transform the known hard problem into yours, in polynomial time, so that yes maps to yes and no maps to no.

CSCE 629 will ask you to write that reduction carefully. This lecture only asks you to refuse the sentence 'it is exponential, so it is NP-complete.' Exponential is a running time. NP-complete is a class.

Two different claims.

In NP

A yes certificate checks quickly

Sorting's decision version is in NP, and also in P

NP-complete

In NP, and as hard as the rest

A reduction, not a feeling

A certificate for a path

1. The question: is there a path of length at most 4 from A to Z.

2. The certificate is the list of vertices. Checking the edges and the length is a scan.

3. Finding the path might be easy on this graph. The certificate definition does not say the problem is hard. It says yes answers are easy to check.

Problem 1. You showed an exponential algorithm. Is the problem NP-complete?

Answer. You have not shown that. You have shown one algorithm is slow. A faster one may exist. NP-completeness needs a reduction from a known NP-complete problem, and a polynomial checker.

Problem 2. Why does CSCE 625 list CSCE 411 as a prerequisite?

Answer. Search and learning sit on graphs, recurrence, and the habit of a bound. 411 is that course. 629 is the graduate version and also belongs on a master's plan. Do not skip 411's table-filling and Dijkstra and then start 625.